

Adult Income Prediction — Final Project Report

Week 1 Capstone — AI & Data Science Internship

1. Problem Definition

The project predicts whether an individual earns more than \$50K per year using the Adult Census dataset, so the prediction can be used to target marketing outreach efficiently. Contacting only likely higher-income prospects reduces wasted outreach while still capturing most of the valuable audience.

F1-score was chosen as the primary success metric because it balances precision (avoiding wasted outreach on unlikely candidates) and recall (not missing genuine high-income prospects). The target for the week was to beat the best hand-written baseline rule ($F1 \approx 0.50$) and reach $F1 > 0.60$ with a trained model.

2. Data Preparation

Data source: `fetch_openml('adult', version=2)`. ~48,842 rows with 14 raw features covering demographics, education, occupation, and hours worked. Missing values, encoded as '?', were converted to NaN and handled with imputation rather than dropped, to avoid losing data.

Feature engineering (6 new features)

- `age_bucket` — 6 age ranges to capture non-linear age effects
- `hours_bin` — part-time / full-time / overtime
- `has_capital_gain` — flag for any investment income
- `log_capital_gain` — log transform to fix skew
- `higher_ed` — flag for `education-num` ≥ 13
- `edu_hours_interaction` — `education-num` $\times$ `hours-per-week`

Preprocessing & validation strategy

Numeric features: median imputation + `StandardScaler`. Categorical features: most-frequent imputation + one-hot encoding. A single stratified 80/20 train/test split (`random_state=42`) was fixed on Day 1 and reused unchanged through Day 5, so the test set was never touched during training or tuning.

3. Model Development

Three baseline models (majority-class, a single education rule, and a multi-feature rule) established a floor. Logistic Regression, Random Forest, and HistGradientBoosting were then trained and compared with 5-fold stratified cross-validation on accuracy, ROC-AUC, and F1-score.

4. Model Tuning

`RandomizedSearchCV` (20 iterations, 5-fold StratifiedKfold, optimizing ROC-AUC) searched each model's key hyperparameters — regularization strength and penalty for Logistic Regression; tree count, depth, and leaf size for Random Forest; learning rate, iterations, depth, and L2 regularization for HistGradientBoosting. The tuned models were refit on the full training set and cross-validated; HistGradientBoosting won with the best mean CV ROC-AUC (0.9269), ahead of Random Forest (0.9157) and Logistic Regression (0.9131), and was selected as the final model. Exact best parameters are recorded in `artifacts/best_params.csv`.

5. Diagnostics

A learning curve (train vs. validation ROC-AUC across training-set sizes) was used to check whether the final model needs more data or is overfitting. Validation curves for Logistic Regression's C and Random Forest's

`max_depth` show the classic underfit → sweet spot → overfit pattern used to sanity-check the chosen hyperparameters.

Out-of-fold probabilities were calibrated with isotonic regression and compared to the raw probabilities using the Brier score; calibration was kept only if it improved the Brier score. A threshold sweep on the training set then selected the F1-optimal classification threshold (see `artifacts/threshold_info.csv`), which is used instead of the default 0.50 cut-off.

6. Final Results

The saved pipeline was re-loaded and re-evaluated once on the untouched test set (`day5_final_project.py`, Task 1), confirming it reproduces the same numbers Day 4 reported and that there is no overlap between train and test row indices.

Metric	Final Model — Held-Out Test Set
Accuracy	0.8686
Precision	0.7226
Recall	0.7318
F1-score	0.7272
ROC-AUC	0.9288
PR-AUC	0.8312
Brier score	0.0875

The final model — HistGradientBoosting — beat every shortlisted candidate on cross-validated ROC-AUC (0.9269 vs. 0.9157 for Random Forest and 0.9131 for Logistic Regression), and its held-out test F1 of 0.7272 comfortably clears the Week 1 target of $F1 > 0.60$ set on Day 1 (baseline $F1 \approx 0.50\text{--}0.55$).

The confusion matrix (`day5_outputs/figs/final_confusion_matrix.png`) breaks these numbers into true/false positives and negatives. False negatives (missed >50K earners) are treated as the costlier error for this marketing use case, which is part of why an F1-optimized threshold — rather than the default 0.50 — was used.

7. Model Interpretation

Permutation importance was computed directly on the saved pipeline using the raw input columns, so the result is valid regardless of which model won or whether it is wrapped in probability calibration. Capital-gain, education-num, hours-per-week, and age are consistently among the strongest predictors, matching real-world income patterns. Marital-status and relationship also carry weight — these are demographic proxies rather than skill signals, so the model is appropriate for aggregate marketing use but should not be used for individual high-stakes decisions without a fairness review.

8. Production Readiness

The trained pipeline — feature engineering, preprocessing, and classifier (plus calibration, if used) — is saved as a single artifact: `artifacts/final_pipeline.joblib`. A `predict_income()` function in `day5_final_project.py` loads this artifact, accepts new raw rows, and returns a probability and a thresholded prediction without repeating any preprocessing manually. It was tested end-to-end on 10 unseen rows from the test set.

9. Limitations & Future Improvements

- The dataset reflects 1994 U.S. Census data; income patterns and the \$50K threshold are outdated for present-day use.
- `fnlwgt` is a census sampling weight, not a personal attribute, and can add noise if the model leans on it.
- Some demographic subgroups are small in this dataset, making their subgroup metrics less reliable — more data per subgroup would help.
- Additional interaction features (e.g. `occupation × hours`) could help separate high-effort/modest-pay roles from genuinely high earners.
- A different, more recent income dataset would be the most impactful future improvement for real-world deployment.

10. Best Parameters

The final model — `HistGradientBoosting` — was tuned with `RandomizedSearchCV` (20 iterations, 5-fold `StratifiedKfold`, optimizing ROC-AUC). The winning configuration:

Hyperparameter	Value	Why it helps
<code>learning_rate</code>	0.1165	Moderate step size — small enough to avoid overshooting, large enough that 87 rounds is sufficient to converge
<code>max_iter</code>	87	Number of boosting rounds; keeps the model from over-iterating and memorizing noise
<code>max_depth</code>	8	Caps how complex/interaction-heavy each individual tree can get, controlling overfitting
<code>l2_regularization</code>	0.0564	A light L2 penalty on leaf values, smoothing predictions toward less extreme outputs

For reference, the runner-up candidates' best parameters:

Model	Best Parameters
Random Forest	<code>max_depth=13, max_features='sqrt', min_samples_leaf=2, n_estimators=183</code>
Logistic Regression	<code>C=0.9847, penalty='l1'</code>

11. Classification Threshold

Threshold	Type	F1-score (train)
0.50	Default	0.7059
0.40	F1-optimized (selected)	0.7254

The threshold was chosen by sweeping candidate cut-offs (0.10 to 0.95 in steps of 0.05) over the model's 5-fold cross-validated out-of-fold probabilities on the training set, and selecting the value that maximized F1-score. 0.40 outperformed the default 0.50, meaning the model is slightly more willing to flag someone as >50K than a naive 50% cut-off would — recovering true positives (genuine high earners) at a small cost in precision, which matches the project's decision that a missed high-income prospect (false negative) is costlier than a wasted contact (false positive).

12. Dependencies / Environment

Captured from the project's actual pip freeze output (only the libraries this project directly uses):

Library	Version
numpy	2.4.0
pandas	2.3.3
scikit-learn	1.8.0
scipy	1.17.1
joblib	1.5.3
matplotlib	3.10.8
seaborn	0.13.2
openml	0.15.1

Full environment can be regenerated at any time with:

```
pip freeze > requirements.txt
```

13. How to Reproduce Training

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Run each day's script in order
# (each depends on files the previous one saves)
python adult_income_prediction.py # Day 1 - baselines
python AIP_Supervised_model.py   # Day 2 - Logistic Regression / Decision Tree
python FE_CV_MC.py               # Day 3 - feature engineering + CV comparison
python model_tuning.py           # Day 4 - hyperparameter search, calibration, saves
artifacts/
python day5_final_project.py      # Day 5 - final validation, interpretation, inference
```

All scripts fix `random_state=42` and use the same stratified 80/20 `train_test_split`, so the same test set is recreated identically every run and stays untouched until Day 5's final evaluation.

14. How to Run Inference

```
from day5_final_project import predict_income
import pandas as pd

# New rows must have the same raw columns as the original Adult dataset
new_rows = pd.DataFrame([
    {
        "age": 37, "workclass": "Private", "fnlwgt": 200000,
        "education": "Bachelors", "education-num": 13,
        "marital-status": "Married-civ-spouse", "occupation": "Exec-managerial",
        "relationship": "Husband", "race": "White", "sex": "Male",
        "capital-gain": 0, "capital-loss": 0, "hours-per-week": 45,
        "native-country": "United-States"
    }
])
```

```

result = predict_income(new_rows)
print(result)
#    probability_gt_50k prediction
# 0                0.71xx    >50K

```

`predict_income()` loads `artifacts/final_pipeline.joblib` and applies feature engineering, imputation, scaling, and one-hot encoding automatically (all baked into the pipeline), returning a probability plus the thresholded label — no manual preprocessing required from the caller.

15. Key Files

File	Description
<code>artifacts/final_pipeline.joblib</code>	The complete trained pipeline (feature engineering + preprocessing + classifier, calibrated if applicable)
<code>artifacts/preprocessor.joblib</code>	The fitted <code>ColumnTransformer</code> alone, saved separately for inspection
<code>artifacts/best_params.csv</code>	Winning hyperparameters for each of the 3 tuned candidate models
<code>artifacts/cv_results.csv</code>	5-fold CV accuracy / ROC-AUC / F1 for each tuned candidate
<code>artifacts/threshold_info.csv</code>	Default vs. F1-optimized classification threshold
<code>artifacts/final_metrics.csv</code>	Day 4's test-set metrics at both thresholds
<code>day5_outputs/tables/final_comparison_table.csv</code>	Day 5's re-validated final test metrics
<code>day5_outputs/tables/feature_importance.csv</code>	Permutation importance, top features
<code>day5_outputs/tables/subgroup_performance.csv</code>	Accuracy / F1 by sex and race
<code>day5_outputs/figs/</code>	Confusion matrix and feature-importance charts
<code>README.md</code>	Full project documentation